

NPTEL Course on "Computer System Optimization for ML"

Lecture #58

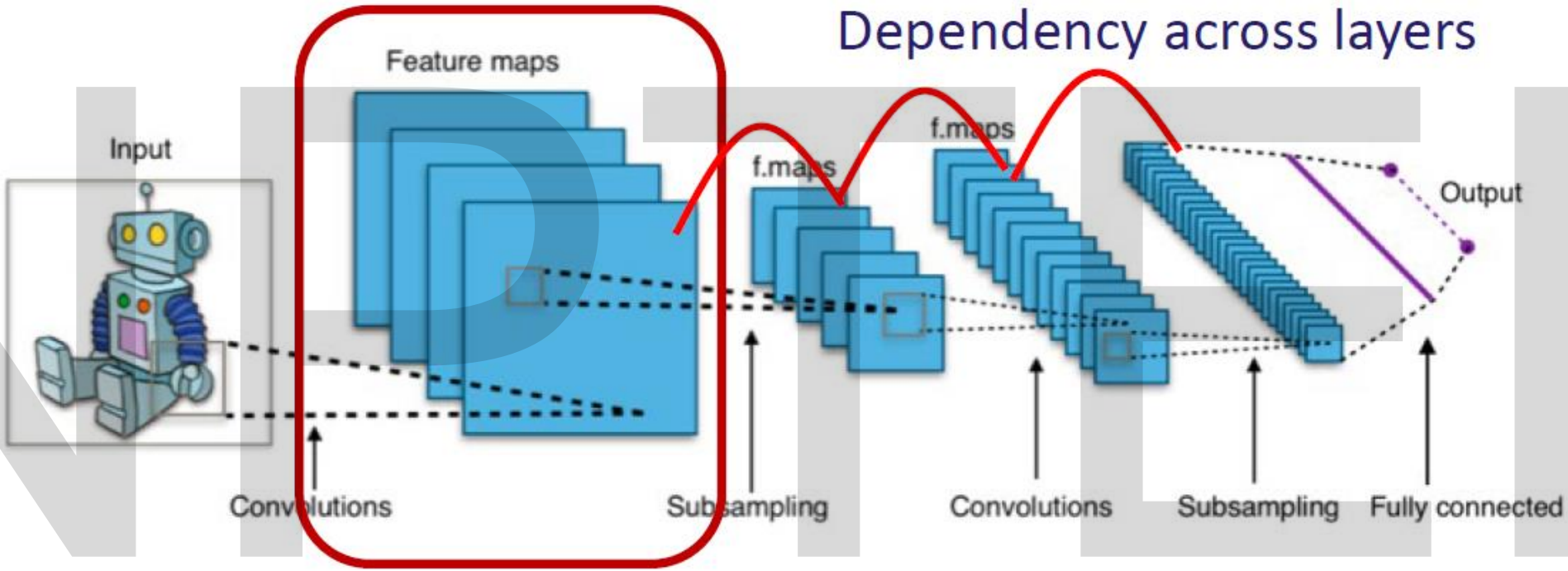
NPTEL

Agenda

- **Acceleration of ML operation with GPUs**

NPTEL

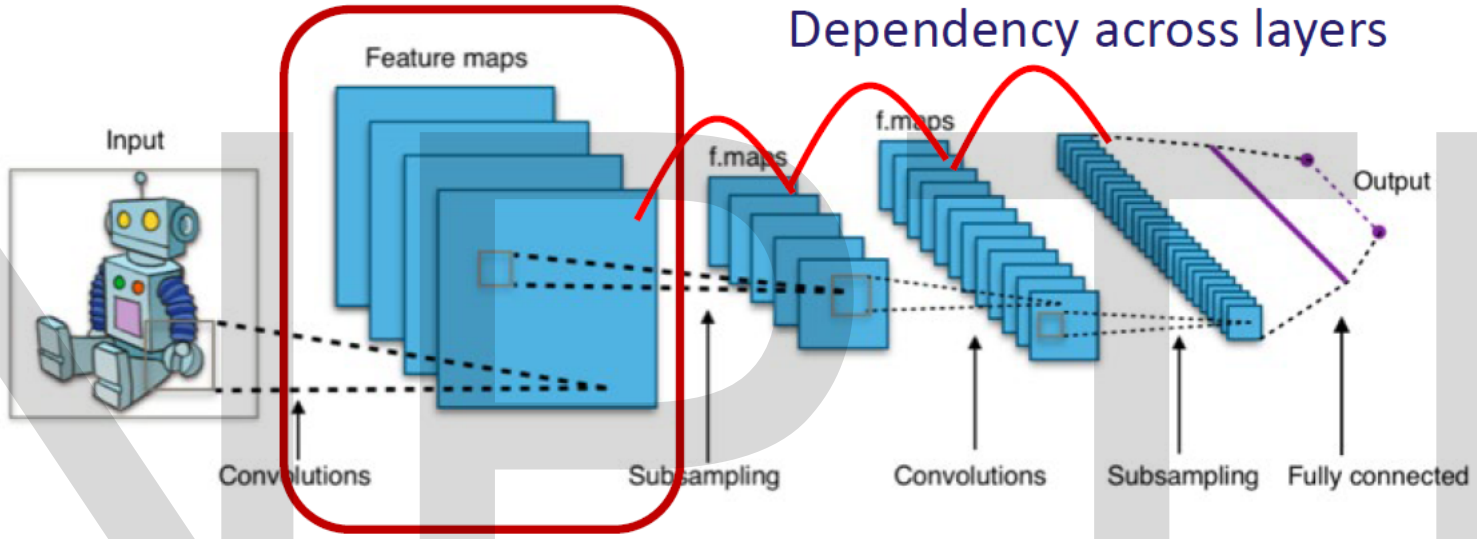
Parallelism for Efficient DNN Execution



Available parallelism within each layer of DNN

INTEL

Parallelism for Efficient DNN Execution

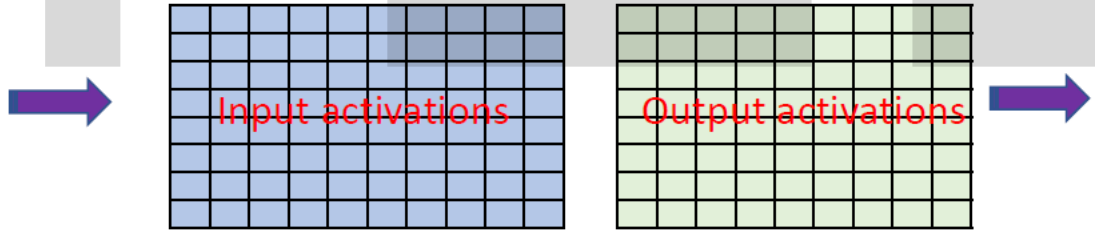


Common computation in linear layers (Conv, FV)

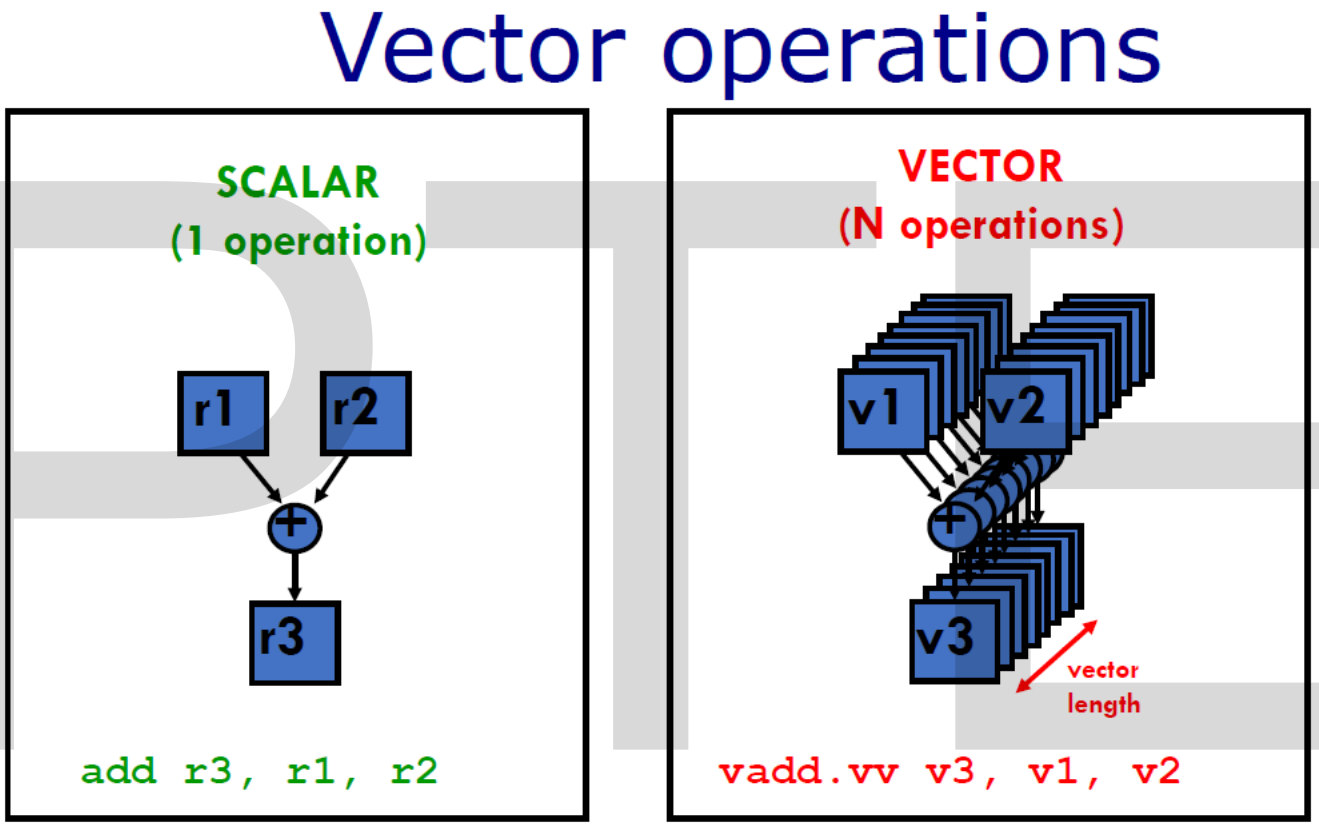


Available parallelism within each layer of DNN

Fundamental computation: Multiply-Accumulate (MAC) [a = a + b*c]



Background on Vector Operations



- Single instruction multiple data
- Vector register file, Vector ALU unit, Vector instruction set

NPTEL

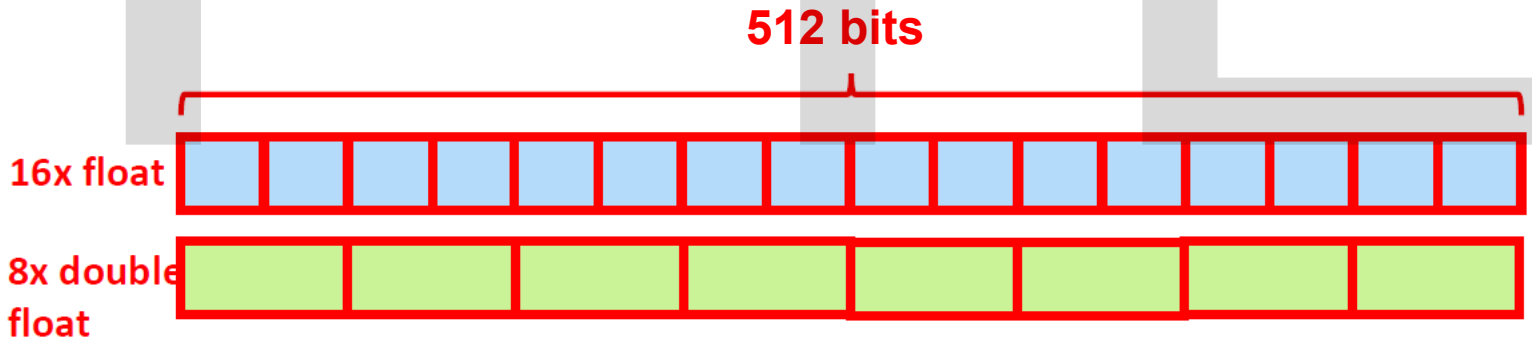
Vector (SIMD) ISA Extension

- Integrated on a CPU core (i.e., extension)
- Example: Intel/AMD's AVX-512
- The CPU core is extended to include vector functional units and vector registers
- Applications must use SIMD instructions/registers
- Typically, invoked through C++ libraries or intrinsicswith specific compiler flags

NPTEL

AVX-512 ISA extension

- 32 vector registers
  - ZMM0 –ZMM31 (different from general purpose x86-64 registers)
- Each vector register is 512 bits wide
  - Each register content can be divided into multiple different data types
  - e.g., quadwords (64-bit), doublewords (32 bit), single (16 bit), float (32 bit) and double float (64 bit)



NPTEL

AVX-512 ISA extension

- Vector instructions
  - Typical vector arithmetic and load/store instructions
  - Scatter/gather instructions
- Vector Fused multiply add (FMA) instruction
  - Performs A = A + B.C
  - Does one rounding

NPTEL

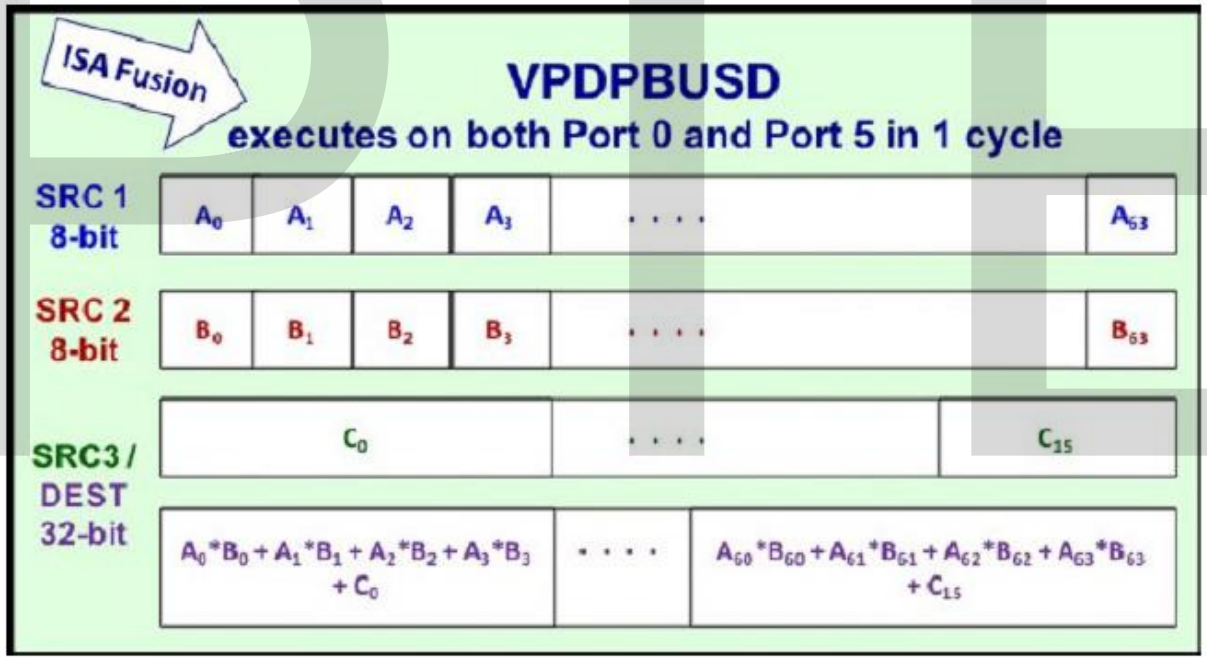
Vector Neural Network Instruction (VNNI)

- Targets higher throughput for inference
- CNN that perform multiplication of two 8-bit (or 16-bit) integers and accumulate the result in a 32-bit integer
  - Lower bit width reduces memory b/w and footprint
  - Better compute throughput and power
- Particularly useful for quantized model
  - v <-a*i, where a is the tensor wide scaling factor and “i” is 8 bit integer
  - a = max |v|/128 for a tensor {v}

NPTEL

Vector Neural Network Instruction (VNNI)

- Introduces a single vector instruction for 8 bit multiply to 32 bit accumulate
- 4X throughput gain



NPTEL

Pros and Cons of SIMD ISA Extension

- Pros
  - Part of CPU core –no new accelerator is needed
  - SIMD ISA extension exploitable by the compiler
  - Often good enough for inference
- Cons
  - Limited Data Level Parallelism (DLP) –not enough for training

NPTEL



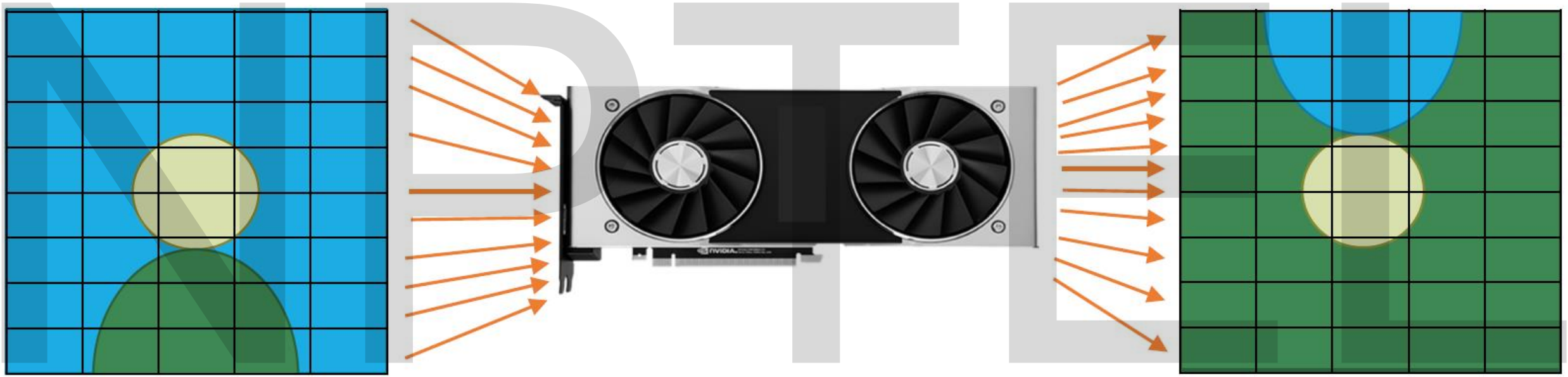
NPTEL

Key Approaches for DLP

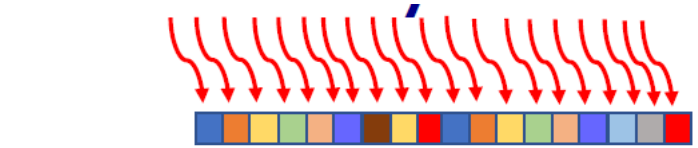
Graphics Processing Unit (GPU)

IISc

Why is GPU used for Parallel Computing

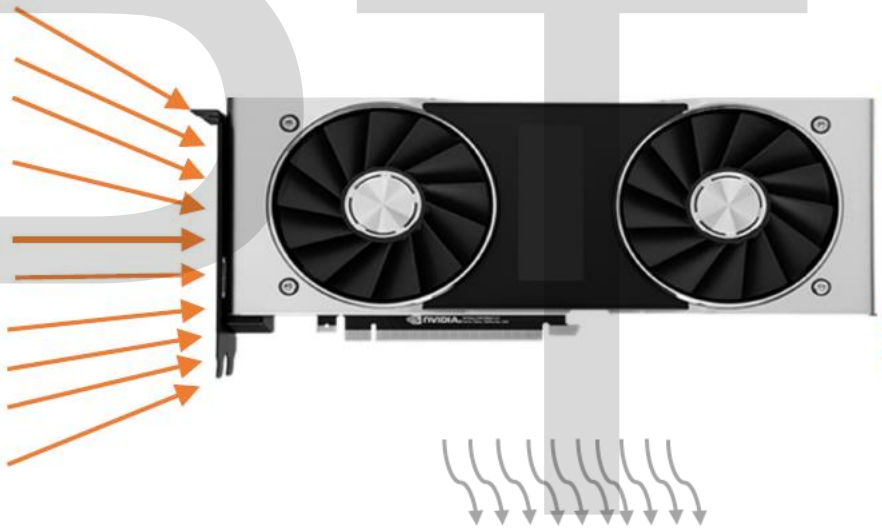


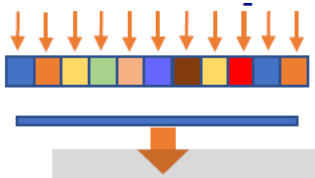
Why is GPU used for Parallel Computing?



Single-instruction multiple data (SIMD)

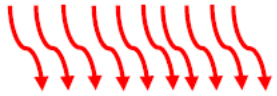
0	71	3	15	88
5	0	8	24	0
1	0	21	0	6
0	23	10	16	0
66	0	23	0	4
35	2	0	30	0
44	0	27	19	0
0	0	22	39	81





Memory coalescing

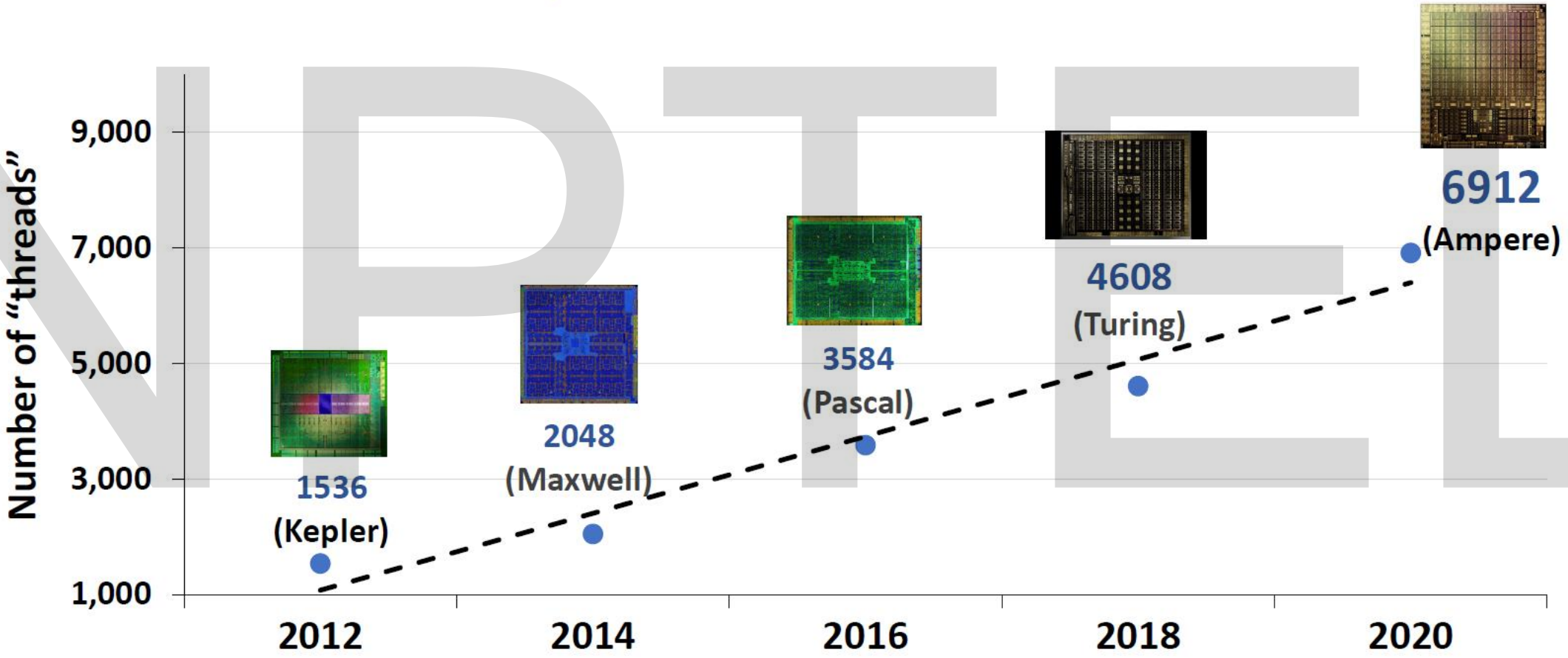
0	33	72	4	13
9	0	66	25	8
36	22	0	13	15
0	23	17	5	0
21	0	1	0	12
6	97	0	45	23
4	0	65	0	43
0	55	42	9	0



Fast hardware context switching

NVIDIA's CUDA (~2007) was one of the first language to harness GPU for compute

Massive Parallel Execution in GPUs



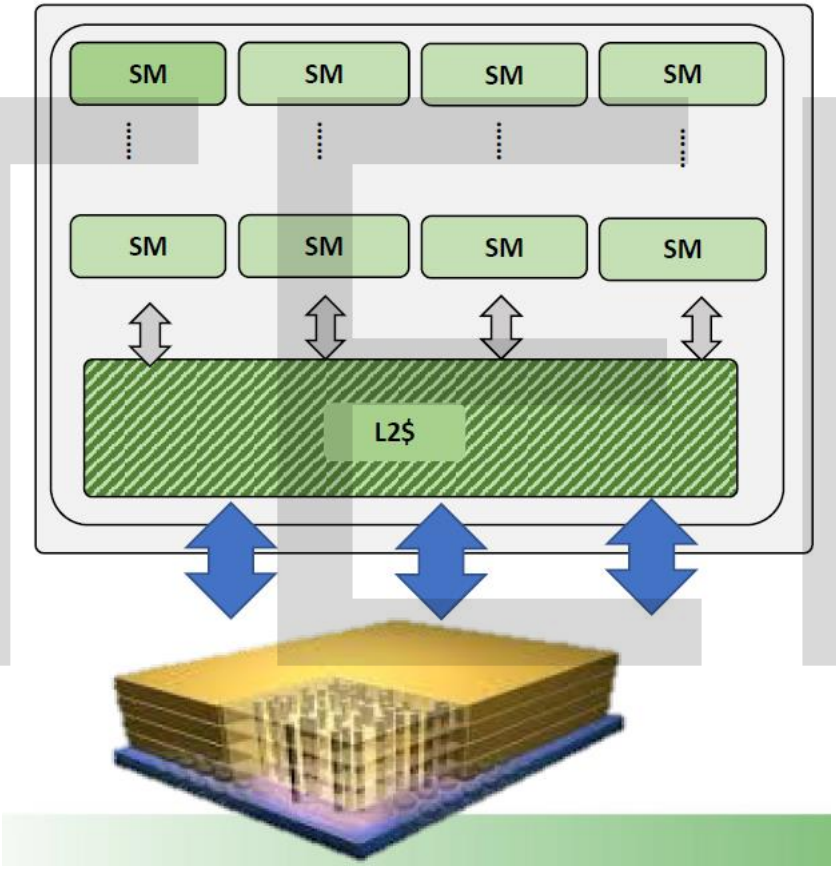
Programming and Execution Model needs to scale to larger GPUs

NPTEL

Execution Hierarchies in GPUs

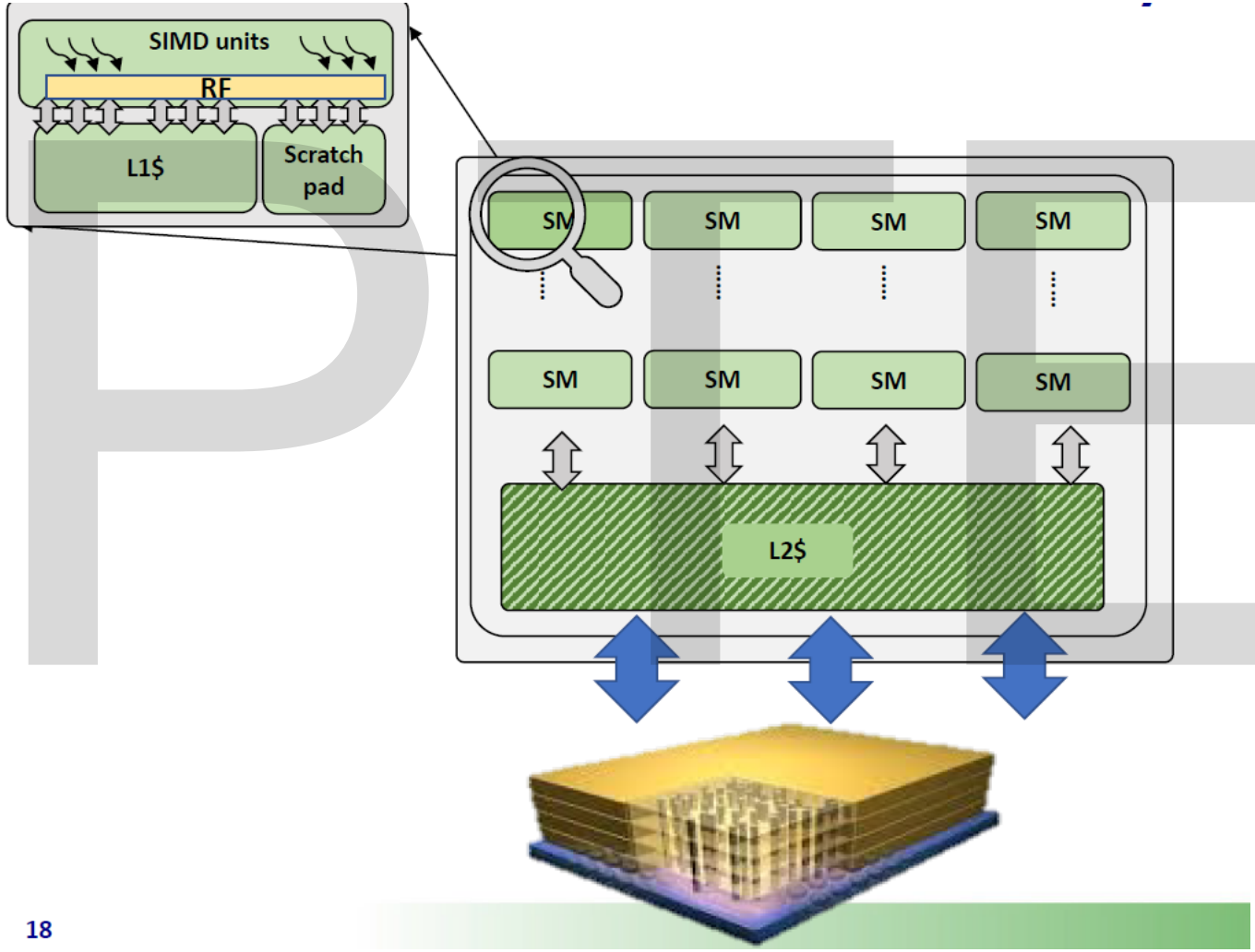


SM -> Symmetric Multiprocessor



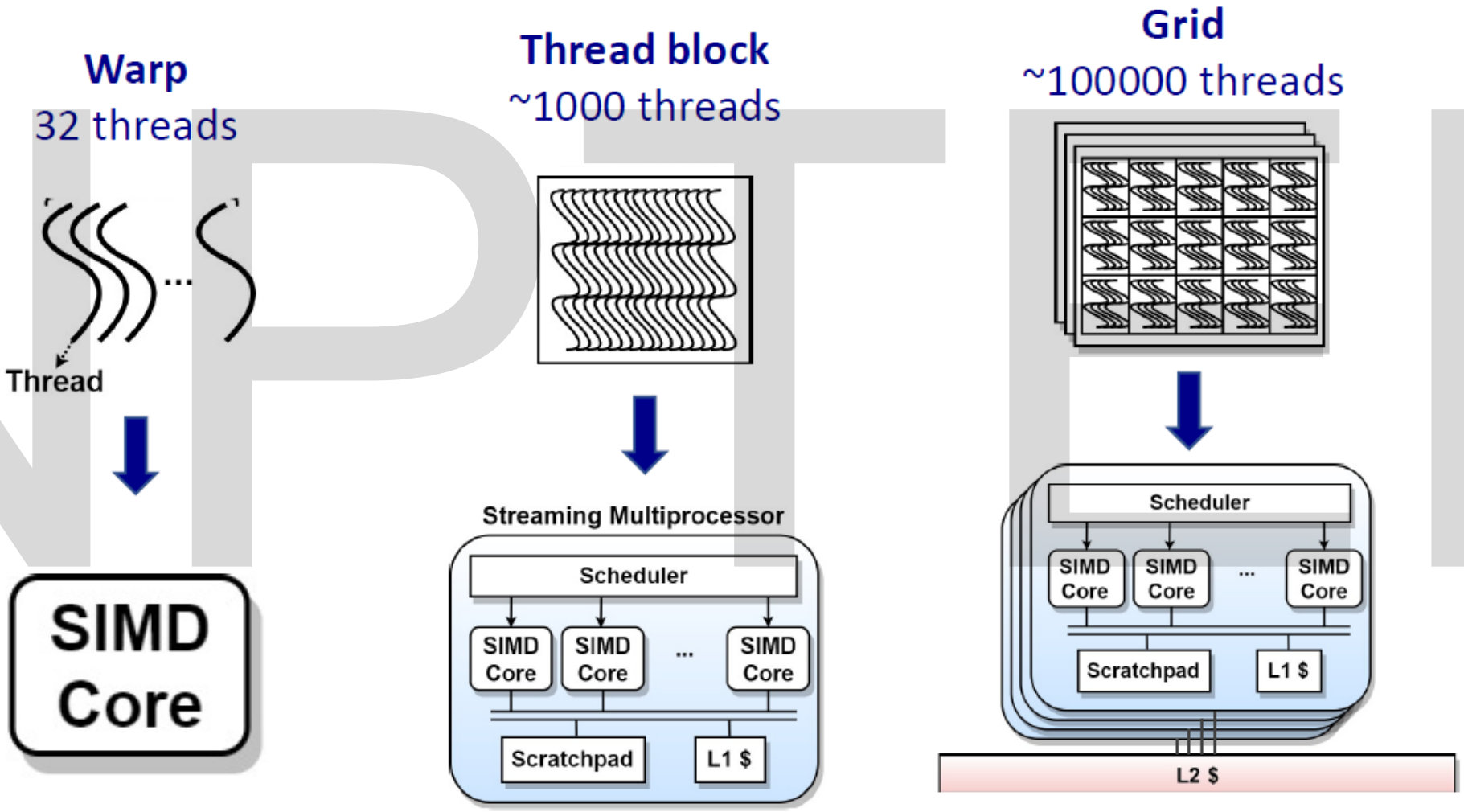
High Bandwidth Memory (HBM)

Execution Hierarchies in GPUs



18

Programming Hierarchy in GPUs



NPTEL

Example CUDA Program

CPU (Host) Code

```
//GPU memory allocation

cudaMalloc((void **)&a, sizeof (int)*m*n)
cudaMalloc((void **)&a, sizeof (int)*n*k)
cudaMalloc((void **)&a, sizeof (int)*m*k)

//Move data from DRAM to GPU memory
cudaMemcpy(a, h_a, sizeof(int)*m*n,
cudaMemcpyHostToDevice)
cudaMemcpy(a, h_a, sizeof(int)*m*n,
cudaMemcpyHostToDevice)

//Launching kernel
matrix_mul<<<dimGrid, dimBlock>>>(a,b,c, m,n,k)

//copy results back to CPU
cudaMemcpy(h_c, c, sizeof(int)*m*k,
cudaMemcpyDeviceToHost)


cudaThreadSynchronize()
```

GPU (Device) Code

```
__global void gpu_matrix_mult(int *a, int * b, int * c, int m,
int n, int k) {

int row = blockIdx.y + blockDim.y + threadIdx.y;
int col =   blockIdx.x + blockDim.x + threadIdx.x;

int sum = 0;


if ( col < k && row < m)

   for (i = 0;  i < n ; i++) {

                sum+ = a[row* n + [I * k + col];

    }

    c[row *k + col ] = sum
}
```

Single Program Multiple Data

Typical DNN Software Stack

TensorFlow

Caffe 2

PyTorch

cuBLAS

cuDNN

cuSOLVER



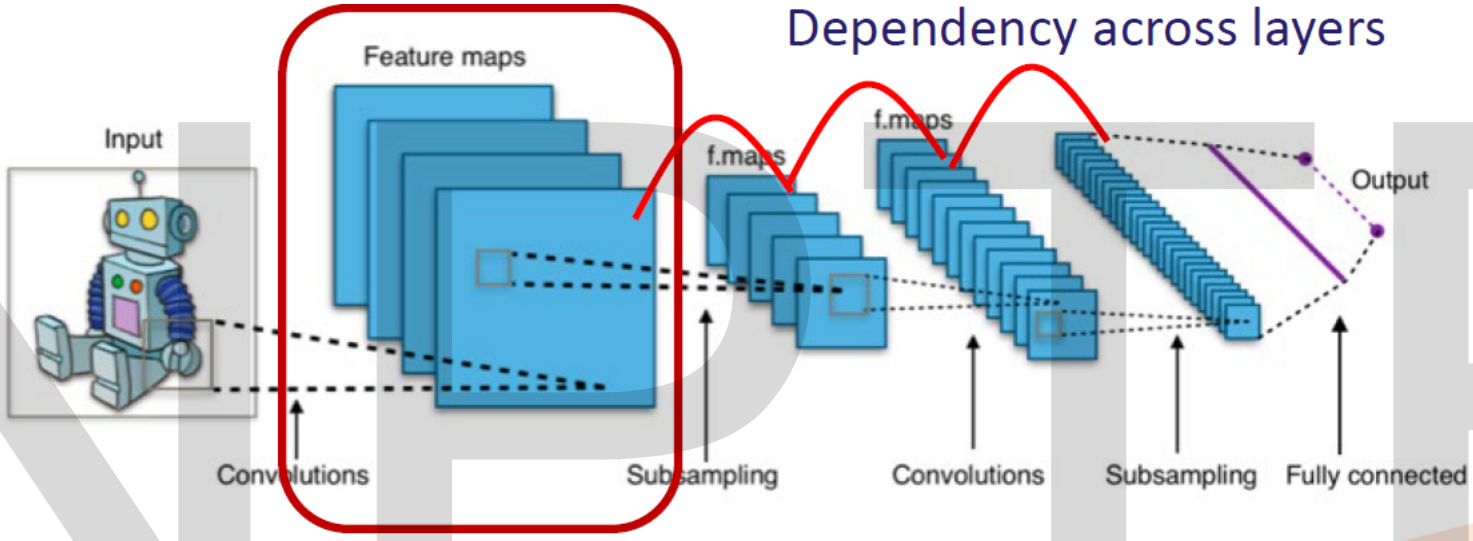
NPTEL

GPU Performance Pitfalls

- Control flow divergence kills performance
  - If/else branches
- Memory access divergence kills performance
  - Uncoalesced memory accesses
- Limited memory capacity
  - Max out at 80GB today at the top end ($$$ GPU)
- Keeping GPU fully utilized isn't easy
  - Can be memory b/w bound

NPTEL

Specializing GPU for DNNs: Tensor Cores

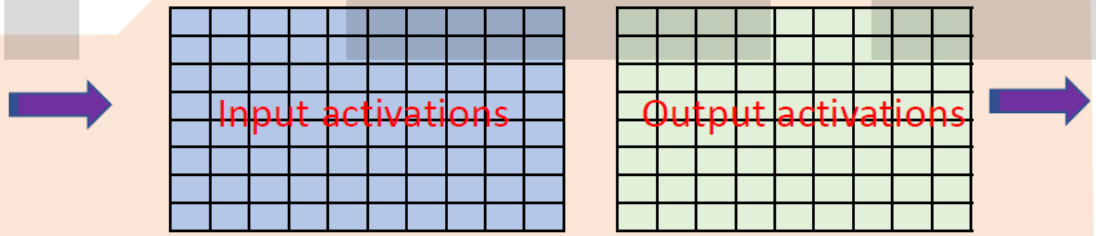


Common computation in linear layers (Conv, FV)

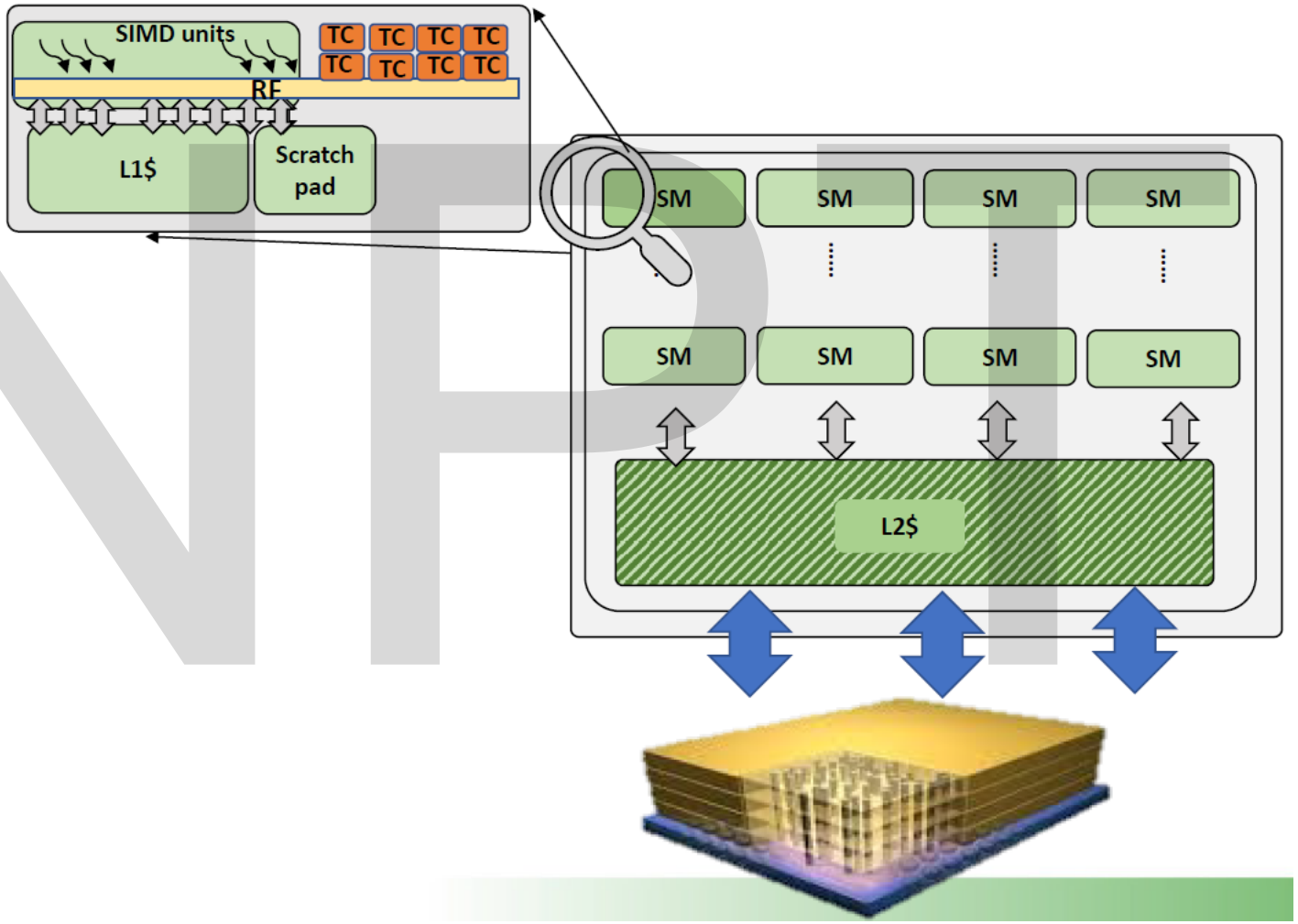


Available parallelism within each layer of DNN

Fundamental computation: Multiply-Accumulate (MAC) [a = a + b*c]



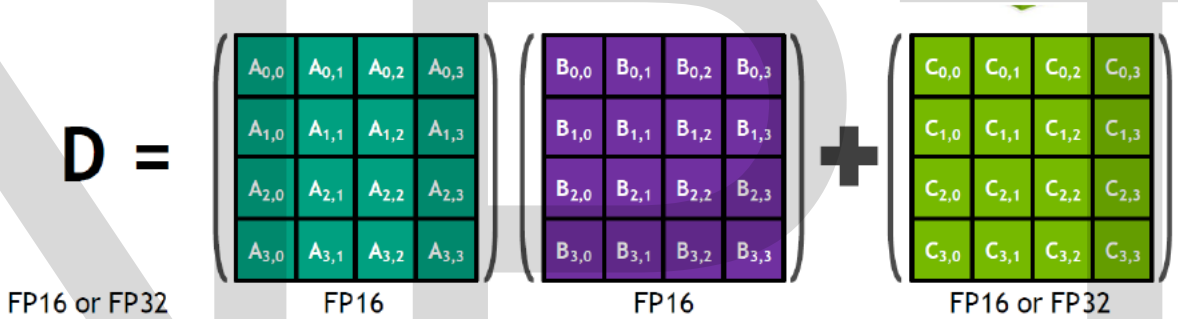
Specializing GPU for DNNs: Tensor Cores

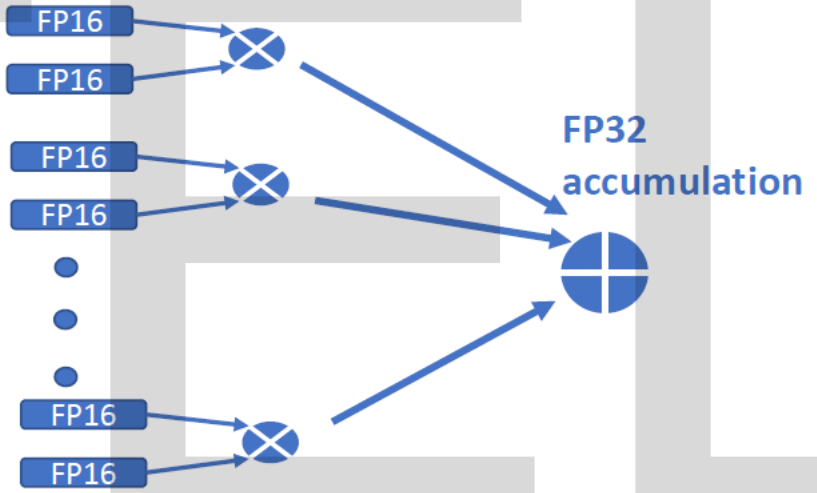


- Accelerator for matrix multiplication
  - Supports mixed precision
  - Warp synchronous
  - Single instruction to invoke Large speedup

NPEL

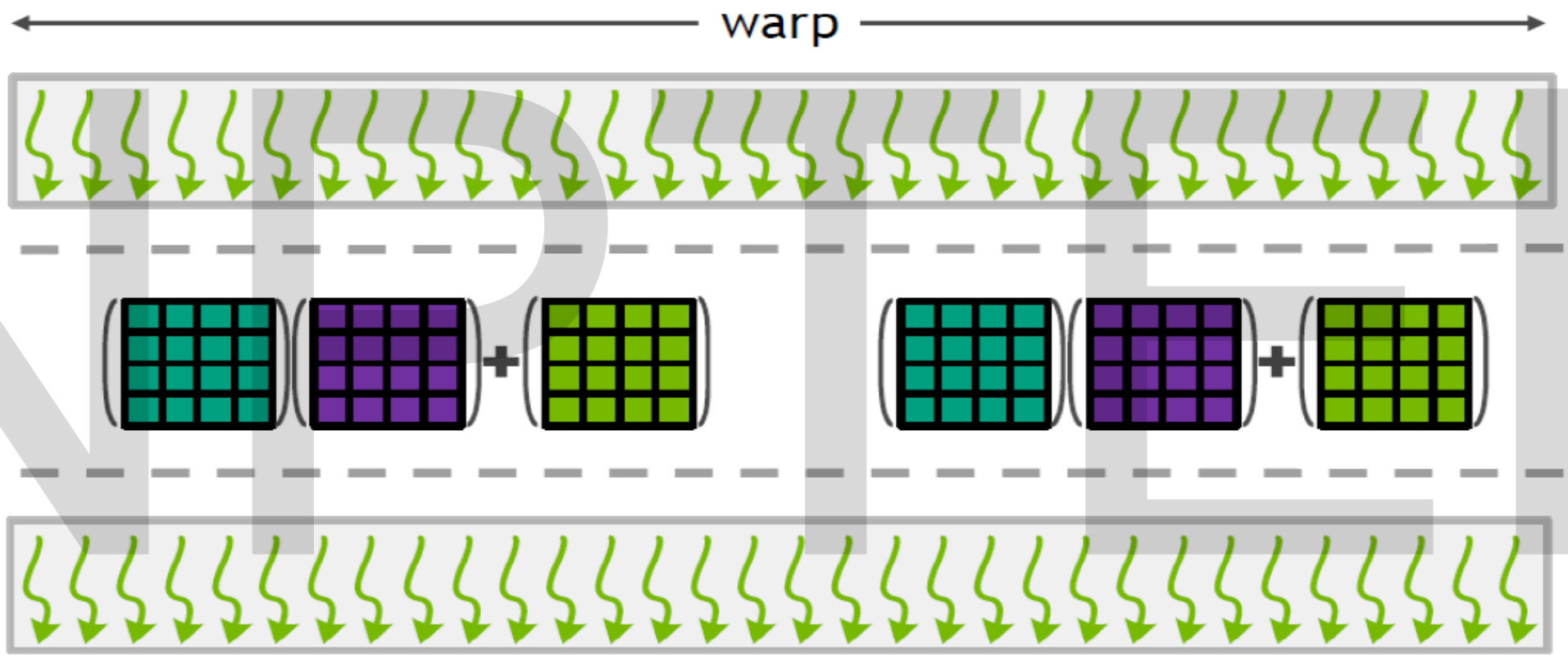
Basic Tensor Core (TC) Operation





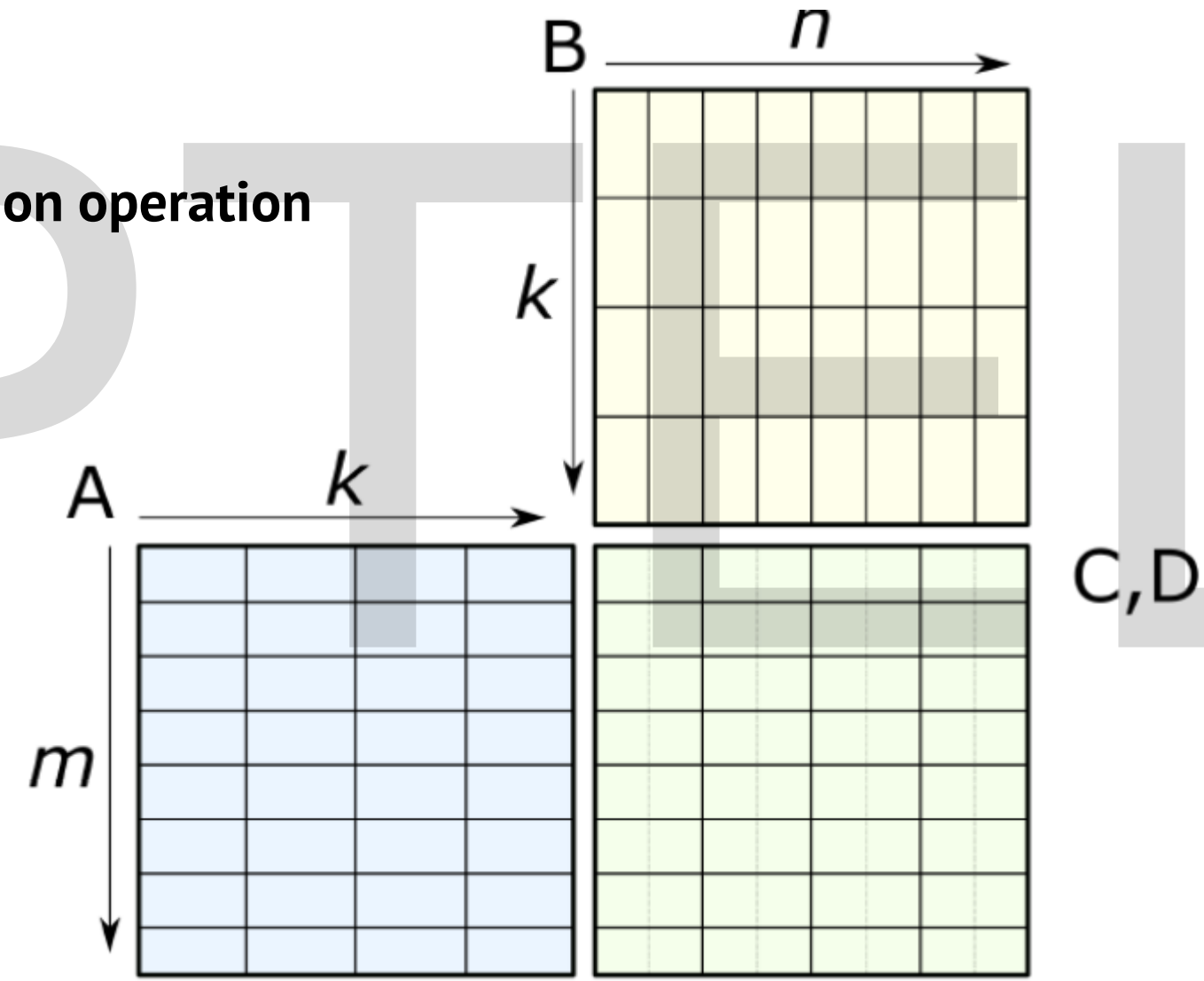
- In Volta architecture (~2017/18), 8 TC per SM, each performing, 64 FP16/FP32 FMA per cycle
- In Ampere architecture (2020), 4 TC per SM, each performing 256 64 FP16/FP32 FMA per cycle

Warp Synchronous Matmul in TC



Work Distribution among Warp Threads

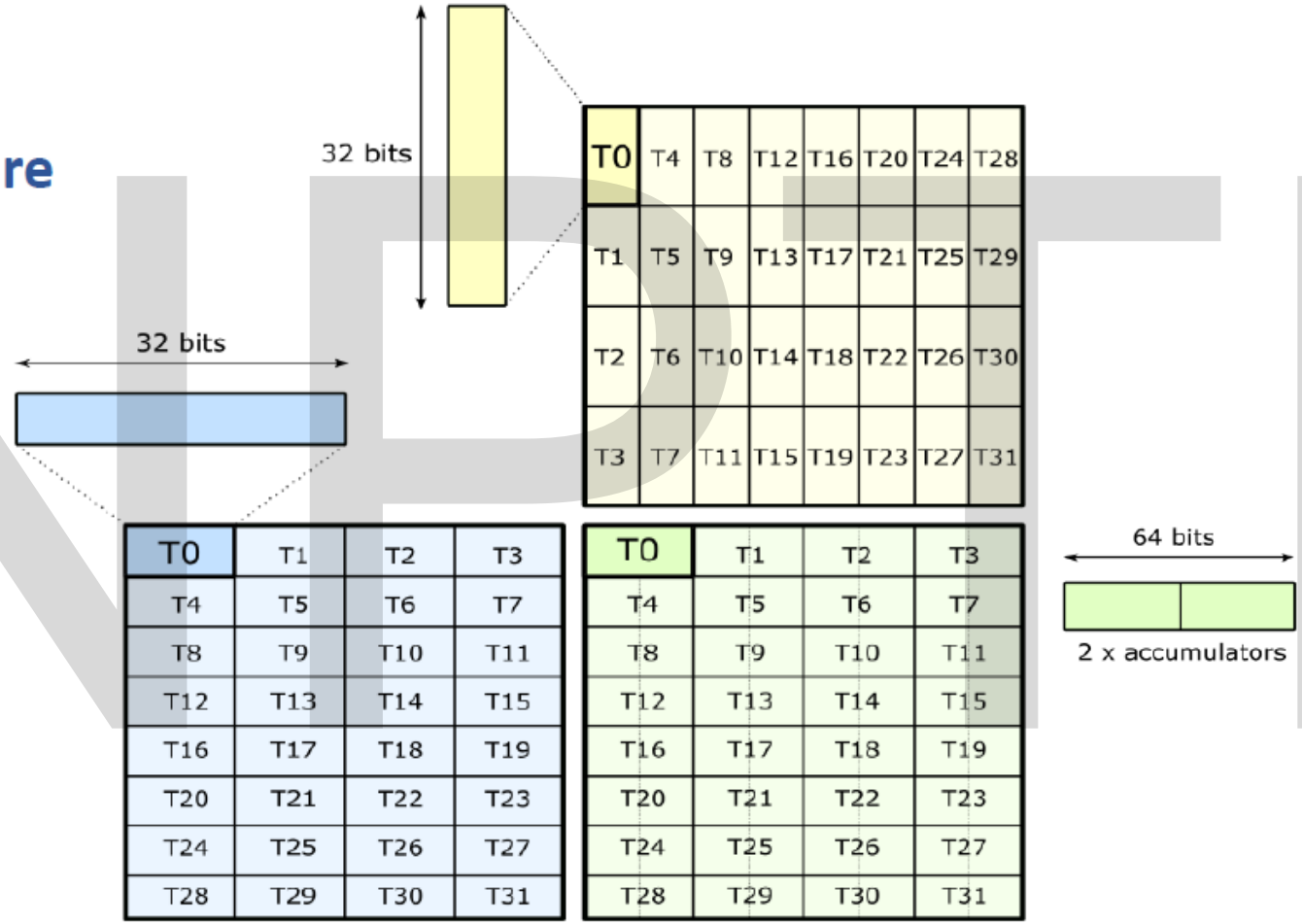
MxNxK matrix multiplication operation



NPTEL

Work Distribution among Warp Threads

Ampere architecture



- wmma instructions take parameters for shape and data types
- shape: m8n4k4
- Data type: f32 f32f64
- Only a limited number of shapes and data types are allowed by the ISA

Speed-up of Matmul with Tensor Cores

PTX	Data Types (A * B + C)	Shape	Speedup on NVIDIA A100 (vs F32 CUDA cores)
mma.sync.m16n8k16 mma.sync.m16n8k8	F16 * F16 + F16 F16 * F16 + F32 BF16 * BF16 + F32	16-by-8-by-16 16-by-8-by-8	16x
mma.sync.m16n8k8	TF32 * TF32 + F32	16-by-8-by-8	8x
mma.sync.m8n8k4	F64 * F64 + F64	8-by-8-by-4	2x
mma.sync.m16n8k32 mma.sync.m8n8k16	S8 * S8 + S32	16-by-8-by-32 8-by-8-by-16	32x
mma.sync.m16n8k64	S4 * S4 + S32	16-by-8-by-64	64x
mma.sync.m16n8k256	B1 ^ B1 + S32	16-by-8-by-256	256x

NPTEL



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)